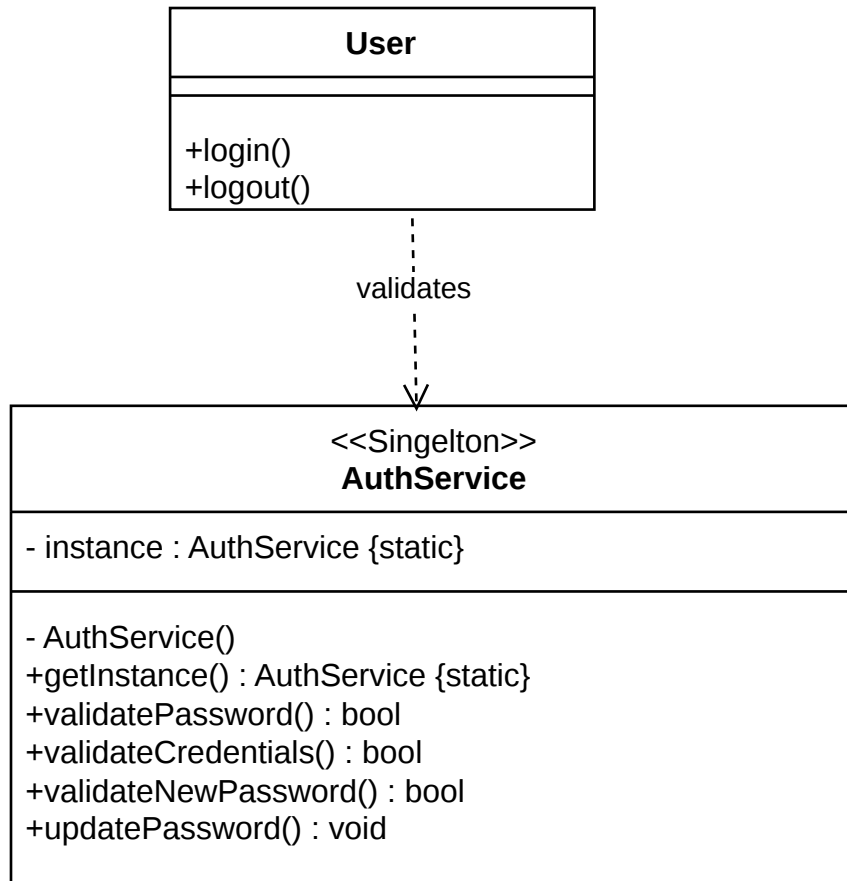


Singleton Design Pattern – AuthService

Xhesika Toska



The Singleton Design Pattern is implemented in the AuthService class to ensure that only one instance of the authentication service exists throughout the entire system. This design pattern is useful for centralized functionalities such as user authentication, password validation, credential verification, and password updates.

In the UML diagram, the AuthService class contains a private static instance (`instance : AuthService {static}`), which stores the single object of the class. The private constructor (`- AuthService()`) prevents other classes from creating new objects directly. Instead, the system accesses the single instance through the public static method `getInstance()`, which always returns the same object.

The User class interacts with AuthService through a dependency relationship (`validates`), meaning that users rely on this service for login validation and authentication-related operations. This implementation improves security, consistency, and resource management, since all authentication processes are handled by one centralized service instance.

Overall, the Singleton pattern ensures better control, maintainability, and efficiency in the authentication system.